

UD5. PSEUDOCÓDIGO. Instrucciones de control.

1. Instrucciones de control.
 - A. Bifurcación incondicional.
 - B. Bifurcación condicional.
 - a) Instrucciones alternativas.
 - I. Alternativa Simple.
 - II. Alternativa doble.
 - III. Alternativa múltiple.
 - b) Instrucciones repetitivas.
 - I. Mientras.
 - II. Repetir.
 - III. Para.
2. Uso de Contador.
3. Uso de Acumulador
4. Uso de Interruptor.

PROG.

Nuria Fuentes Pérez



1. Instrucciones de control.

Son instrucciones que no realizan trabajo efectivo alguno salvo alterar el orden normal de ejecución de las instrucciones de un programa.

Bifurcación incondicional

sin evaluar ningún tipo de expresión. Instrucciones de salto.

Bifurcación condicional

Se puede alterar tras la evaluación de expresiones lógicas, estaríamos hablando de instrucciones de bifurcación condicional

1. Instrucciones de control.

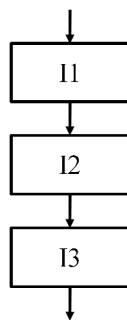
La programación estructurada está basada en tres estructuras algorítmicas básicas:

- **Secuencial:** Ejecución de una instrucción detrás de otra en el orden descrito. Secuencia normal de ejecución de un algoritmo.
- **Alternativa:** alteran la secuencia normal de ejecución de un algoritmo en función de la toma de una decisión (evaluación de una condición). Pudiendo ejecutarse, estructuras secuenciales distintas según la evaluación de la expresión booleana.
- **Repetitiva:** alteran la secuencia normal de ejecución de un algoritmo haciendo posible que un grupo de instrucciones se ejecuten más de una vez de forma consecutiva.

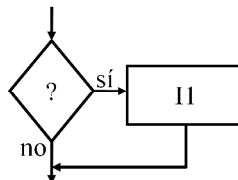
1. Instrucciones de control.

La programación estructurada está basada en tres estructuras algorítmicas básicas:

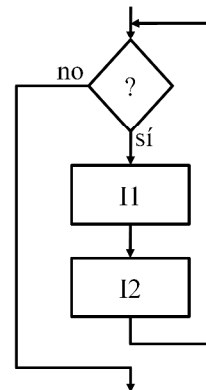
Secuencial



Alternativa o
De selección



Repetitiva
o de iteración



1. Instrucciones de control.

B.a) Instrucciones Alternativas.

➤ Alternativa

Son aquellas instrucciones que alteran la secuencia normal de ejecución de un algoritmo en función de la toma de una decisión. (evaluación de expresión booleana)

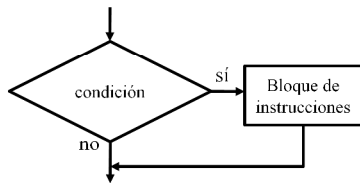
- I. Alternativa simple.
- II. Alternativa doble.
- III. Alternativa múltiple.

1. Instrucciones de control.

B.a).I Instrucciones Alternativas.

➤ Alternativa simple

Dependiendo de la evaluación de una condición (expresión booleana) se ejecutará un bloque de instrucciones o nada.

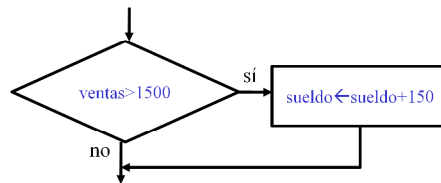


```
Si <condición> entonces
//Bloque instrucciones
Fin Si
```

1. Instrucciones de control.

B.a).I Instrucciones Alternativas.

➤ Alternativa simple: ejemplo



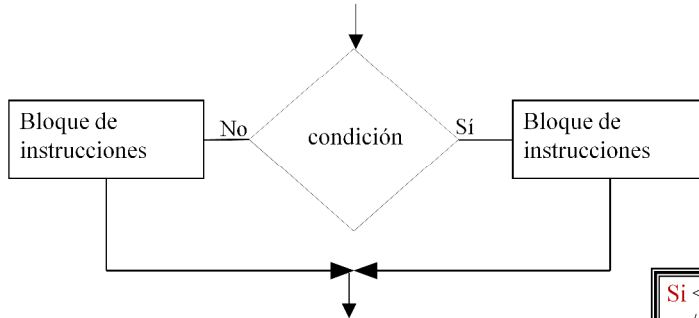
Si ventas > 1500 entonces
sueldo ← sueldo + 150
Fin Si

1. Instrucciones de control.

B.a).II Instrucciones Alternativas.

➤ Alternativa doble

Dependiendo de la evaluación de una condición (expresión booleana) se ejecutará un bloque de instrucciones u otro.



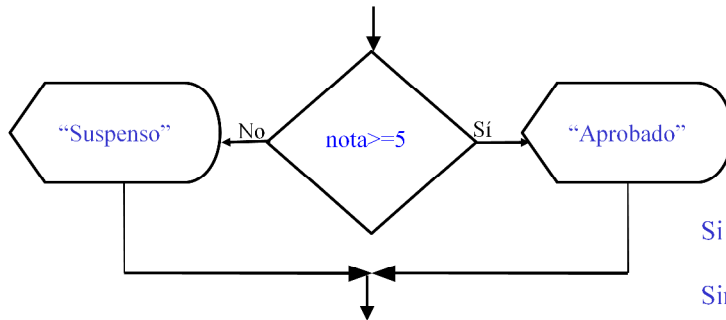
```
Si <condición> entonces
    //Bloque instrucciones
Sino
    //Bloque instrucciones
Fin Si
```


1. Instrucciones de control.

B.a).II Instrucciones Alternativas.

➤ Alternativa doble: ejemplo

Dada una nota, mostrar si está aprobada o suspensa



Si nota >= 5 entonces
Escribir "Aprobado"
Sino
Escribir "Suspenso"
Fin Si

1. Instrucciones de control.

B.a).III Instrucción alternativa múltiple.

➤ Alternativa múltiple

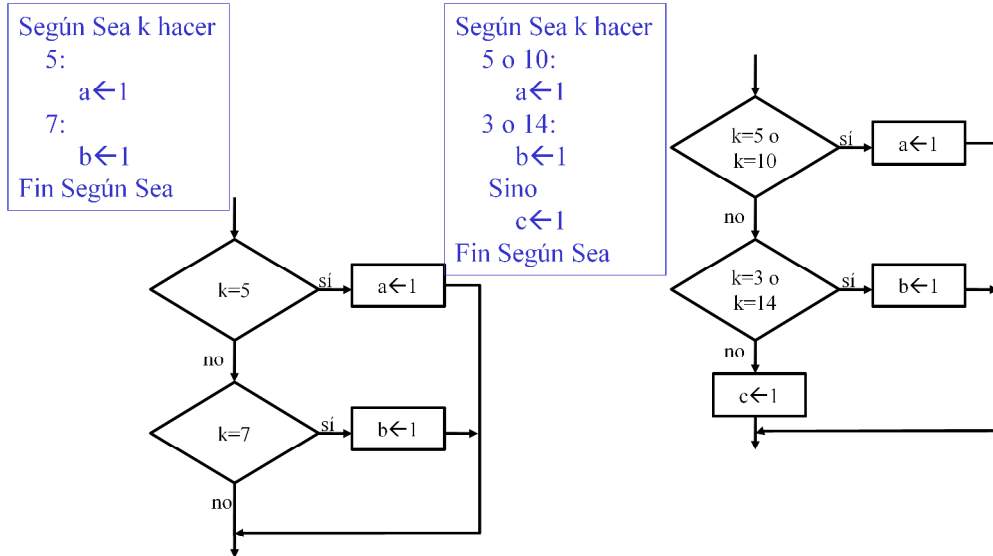
```
Según Sea <nVable> hacer
  <valor1>[ o <valor2>... ] :
    //Bloque de instrucciones
  <valor3>[ o <valor4>... ] :
    //Bloque de instrucciones
  ...
[Sino
  //Bloque de instrucciones]
Fin Según Sea
```

Solo se
compara en
igualdad

Pueden estar
limitado los tipos
de datos

1. Instrucciones de control. B.a).III Instrucción alternativa múltiple.

➤ Alternativa múltiple: ejemplos



PROG.

Nuria Fuentes Pérez



1. Instrucciones de control.

B.b) Instrucciones Repetitivas.

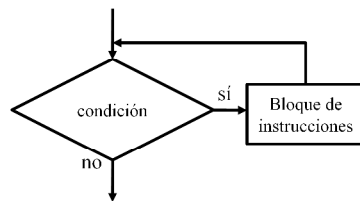
➤ Repetitivas

Son aquellas instrucciones que alteran la secuencia normal de ejecución de un algoritmo haciendo posible que un grupo de instrucciones se ejecuten más de una vez de forma consecutiva.

- I. Mientras
- II. Repetir
- III. Para

1. Instrucciones de control. B.b).I Instrucciones Repetitivas.

➤ Mientras



```
Mientras <condición> hacer
//Bloque instrucciones
Fin Mientras
```

Condición de permanencia o de salida????

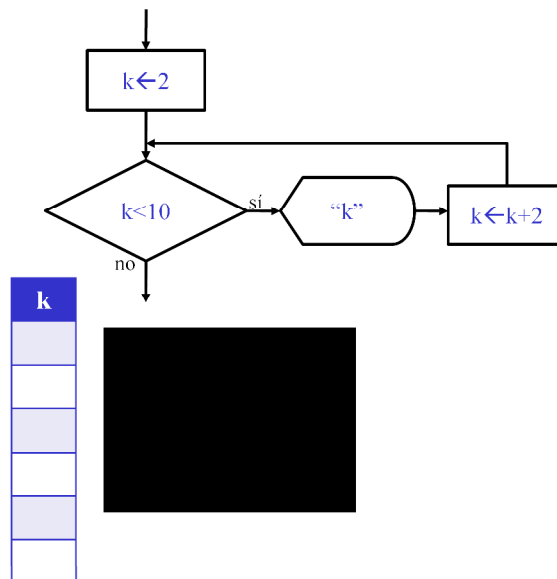
Pregunta antes de entrar????

El bloque de instrucciones se ejecuta mínimo 0 o 1 vez????

1. Instrucciones de control. B.b).I Instrucciones Repetitivas.

➤ Mientras: ejemplo

$k \leftarrow 2$
Mientras $k < 10$ hacer
 Escribir "k"
 $k \leftarrow k + 2$
Fin Mientras



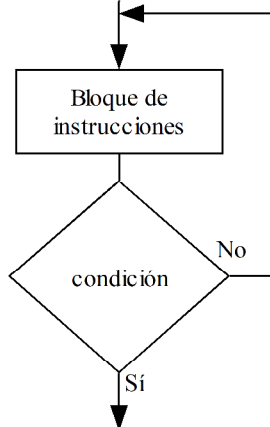
PROG.

Nuria Fuentes Pérez

1. Instrucciones de control.

B.b).II Instrucciones Repetitivas.

➤ Repetir



```
Repetir
//Bloque instrucciones
Hasta <condición>
```

Condición de permanencia o de salida???

Pregunta antes de entrar???

El bloque de instrucciones se ejecuta mínimo 0 o 1 vez???

PROG.

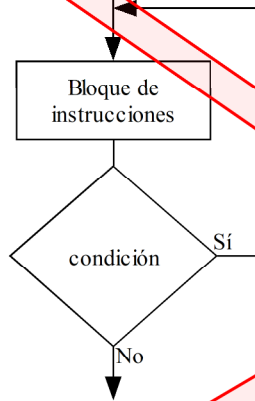
Nuria Fuentes Pérez



1. Instrucciones de control.

B.b).II Instrucciones Repetitivas.

➤ Repetir



Repetir
//Bloque instrucciones
Mientras <condición>

No existe en
pseudocódigo

Condición de permanencia o de salida????

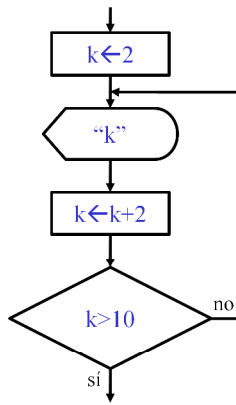
Pregunta antes de entrar????

El bloque de instrucciones se ejecuta mínimo 0 o 1 vez????

1. Instrucciones de control.

B.b).II Instrucciones Repetitivas.

➤ Repetir: ejemplo



$k \leftarrow 2$
 Repetir
 Escribir "k"
 $k \leftarrow k+2$
 Hasta $k > 10$

k



$k \leftarrow 2$
 Repetir
 Escribir "k"
 $k \leftarrow k+2$
 Mientras $k \leq 10$

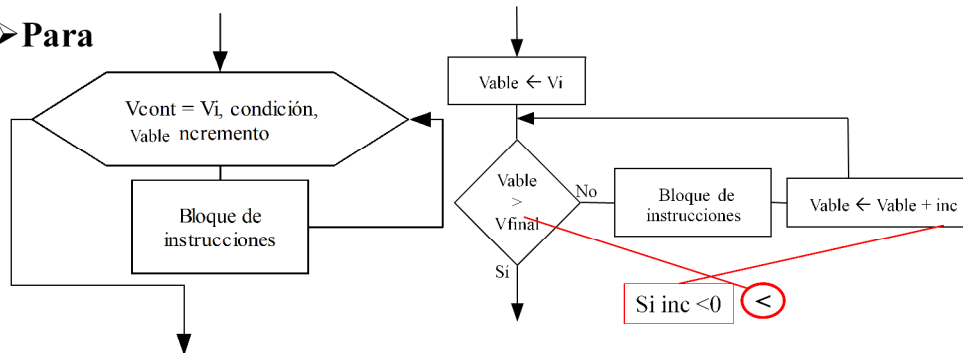
PROG.

Nuria Fuentes Pérez

1. Instrucciones de control.

B.b).III Instrucciones Repetitivas.

➤ Para



Para <Vable> **de** <valor inicial> **a** <valor final> [**con incremento** <Inc>] **hacer**
 //Bloque instrucciones
Fin Para

Condición de permanencia o de salida????

El bloque de instrucciones se ejecuta mínimo 0 o 1 vez????

PROG.

Nuria Fuentes Pérez



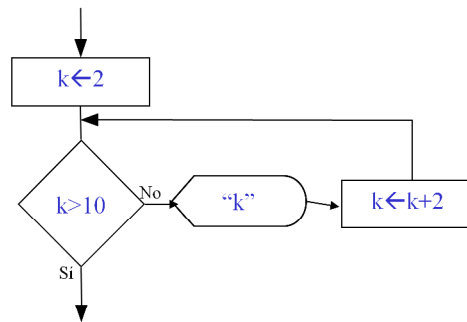
1. Instrucciones de control.

B.b).III Instrucciones Repetitivas.

➤ Para:ejemplo

Para k de 2 a 10 con incremento 2 hacer
Escribir "k"
Fin Para

k



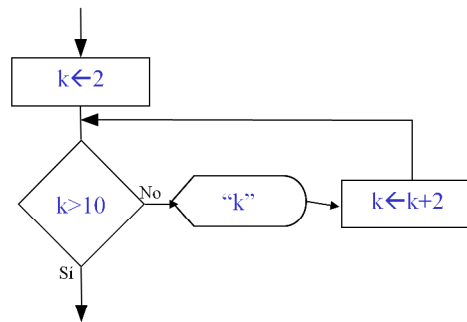
1. Instrucciones de control.

B.b).III Instrucciones Repetitivas.

➤ Para:ejemplo

Para k de 2 a 10 con incremento 2 hacer
Escribir "k"
Fin Para

k



2. Uso de Contador

Es una **variable numérica entera** que está destinada a contener un valor que se irá incrementando o decrementando en una cantidad **fija**. Se usa para contar.

Todo contador se usa en dos pasos:

❑ **Inicializar:** asignamos el valor inicial del contador.

$\text{<variableContador>} \leftarrow \text{<valorIncial>}$

❑ **Incrementar/decrementar:** contamos.

○ Si se cuenta hacia adelante

$\text{<variableContador>} \leftarrow \text{<variableContador>} + \text{<incremento>}$

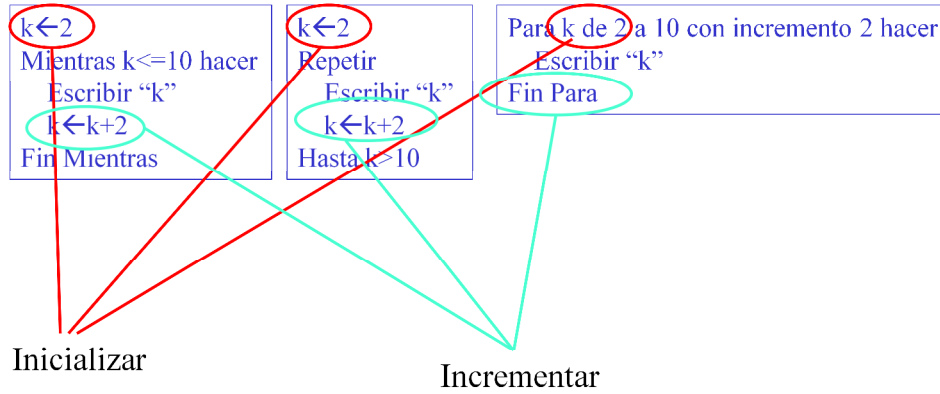
○ Si se cuenta hacia atrás

$\text{<variableContador>} \leftarrow \text{<variableContador>} - \text{<incremento>}$

El incremento debe ser una cantidad fija.

2. Uso de Contador

➤ EJEMPLOS



En los tres ejemplos k es un contador

2. Uso de Acumulador

Es una **variable numérica** que está destinada a contener un valor que se irá incrementando o decrementando en una cantidad **variable**. Se usa para calcular totales o subtotales. (Σ sumatorio en matemáticas)

Todo acumulador se usa en dos pasos:

- ❑ Inicializar: asignamos el valor inicial del acumulador.

`<variableAcumulador> ← <valorIncial>`

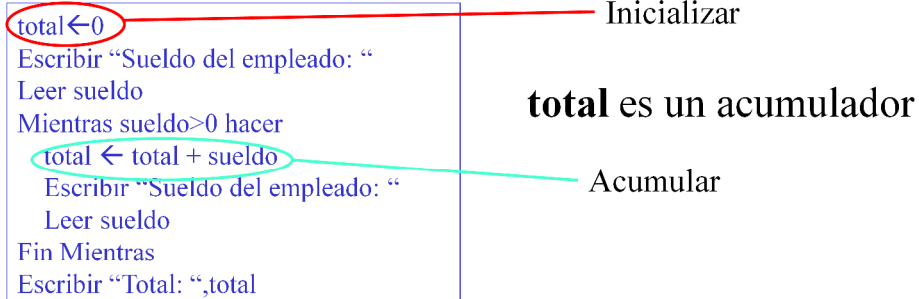
- ❑ Acumular: sumamos o restamos una cantidad variable.

`<variableAcumulador> ← <variableAcumulador> ± <incremento>`

El incremento debe ser una cantidad variable y se irá incrementado o decrementando dependiendo de si el incremento se suma o se resta.

2. Uso de Acumulador

➤ EJEMPLOS



También es una acumulador si es en producto

$\langle \text{variableAcumulador} \rangle \leftarrow \langle \text{variableAcumulador} \rangle * \langle \text{incremento} \rangle$

3. Uso de Interruptor

Un interruptor (conmutador, indicador, flag, bandera o switch) es una variable que sólo pueden tomar dos únicos valores considerados como lógicos y opuestos entre sí (1 y 0, 1 y -1, VERDADERO y FALSO, “on” y “off”). Las variables booleanas son interruptores aunque también se puede implementar un switch con una variable numérica que tome los valores 1 y 0 o 1 y -1, o también con una variable alfanumérica que tome los valores “on” y “off”.

Su objetivo es:

- 1) Poder comprobar en un determinado lugar del programa un suceso que ocurrió o no en algún punto anterior del mismo.
- 2) realizar dos acciones diferentes que se ejecuten de forma alternativa en una repetitiva.

3. Uso de Interruptor

➤ EJEMPLOS

```
esMayorQue3 ← FALSO
Escribir "Número: "
Leer x
Si x > 3 entonces
    esMayorQue3 ← VERDADERO
Fin Si
Escribir "Número: "
Leer x
Si x > 3 y esMayorQue3 entonces
    Escribir "Ambos son mayores que 3"
Fin Si
```

Diagram illustrating the use of the interruptor `esMayorQue3`:

- `esMayorQue3 ← FALSO` is labeled "Inicializar" (Initialize).
- `esMayorQue3 ← VERDADERO` is labeled "Cambia interruptor" (Change interruptor).

esMayorQue3 es un interruptor

3. Uso de Interruptor

➤ EJEMPLOS

```
sumaTres ← 0
multiplicaTres ← 0
esSuma ← VERDADERO
Para grupo de 1 a 5 hacer
  Para i de 1 a 3 hacer
    Escribir "Número: "
    Leer x
    Si esSuma entonces
      sumaTres ← sumaTres + x
    Sino
      multiplicaTres ← multiplicaTres + x
    Fin Si
  Fin Para
  esSuma ← no esSuma
Fin Para
```

Inicializar

Cambia interruptor

esSuma es un interruptor

PROG.

Nuria Fuentes Pérez